КАК Я могу применить rag к моделям через апи?

Чтобы применить RAG (Retrieval-Augmented Generation) через API, вы можете использовать **либо готовые решения от провайдеров (где RAG встроен в API), либо построить собственный конвейер, интегрируя API разных сервисов**. Хабр +1

1. Готовые решения через API (Managed RAG)

Некоторые провайдеры предоставляют «RAG как услугу». Вам достаточно загрузить документы, а API само позаботится о поиске и генерации ответа.

- **OpenAI Assistants API**: Вы создаете ассистента и включаете инструмент `file_search`. Вы загружаете файлы, а API автоматически индексирует их и использует при ответах.
- **Yandex Cloud (YandexGPT + Foundation Models)**: Позволяет подключить внешние источники данных (базы знаний) напрямую к модели через сервисы облака. Yandex Cloud +1

2. Создание собственного RAG через API (Custom RAG)

Это классический путь, который дает полный контроль над качеством. Вам понадобятся три типа API:

Шаг A: Подготовка и Векторизация (Embedding API)

Вам нужно превратить текст документов в числа (векторы), чтобы компьютер мог искать смысл, а не просто слова.

- **Как сделать**: Отправляете куски текста (chunks) в API (например,

Задайте вопрос

2. Отправляете этот вектор в API векторной базы данных, чтобы найти наиболее похожие (релевантные) куски текста из ваших документов. Хабр +1

Шаг C: Генерация (Completion API)

Вы формируете итоговый запрос (промпт) для LLM, добавляя в него найденные данные.

- **Пример промпта**: *"Используй этот текст: [Найденные данные], чтобы ответить на вопрос: [Вопрос пользователя]"*.
- **Действие**: Отправляете этот расширенный промпт в Chat Completions API (OpenAI, Anthropic, DeepSeek и др.). Reddit +1

Резюме: Сравнение подходов

Метод	Сложность	Контроль	Инструменты
Assistants API	Низкая	Низкий	OpenAI
Библиотеки (LangChain/LlamaIndex)	Средняя	Высокий	LangChain + любой LLM API
Полностью свое (Pure API)	Высокая	Максимальный	Python/JS + Curl к API провайдеров

Хотите, я составлю **пример кода на Python** для реализации простейшего RAG через OpenAI API?

Да

Для реализации собственного RAG-конвейера на Python обычно используют связку **OpenAI API** (для генерации и эмбеддингов) и простую библиотеку для поиска (например, **FAISS** или **ChromaDB**).

Ниже приведен минимальный рабочий пример того, как это работает «под капотом» без использования сложных фреймворков вроде LangChain. Хабр +2

1. Установка библиотек

Вам понадобятся официальный клиент OpenAI и библиотека для векторного поиска:

bash

```
pip install openai faiss-cpu numpy
```

Используйте код с осторожностью.

Режим ИИ Все Видео Покупки Картинки Ещё

```
# Инициализация клиента
client = OpenAI(api_key="ВАШ_API_KEY")

# 1. Наша "База знаний" (простые текстовые документы)
documents = [
    "Облачные серверы компании X стоят 500 рублей в месяц.",
    "Техподдержка работает круглосуточно по номеру 8-800...",
    "Скидка 10% предоставляется при оплате за год."
]

def get_embedding(text):
    """Превращает текст в вектор (embedding) через API"""
    response = client.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    return response.data[0].embedding

# 2. Индексация: превращаем документы в векторы и сохраняем в F
doc_embeddings = np.array([get_embedding(doc) for doc in documer
index = faiss.IndexFlatL2(doc_embeddings.shape[1])
index.add(doc_embeddings)

def rag_query(user_question):
    # А) Векторизуем вопрос пользователя
    question_embedding = np.array([get_embedding(user_question)]

    # Б) Поиск: находим самый похожий документ (Top-1)
    D, I = index.search(question_embedding, k=1)
    relevant_context = documents[I[0][0]]

    # В) Генерация: отправляем вопрос + контекст в LLM
    prompt = f"Используй только этот текст: '{relevant_context}'

    response = client.chat.completions.create(
        model="gpt-4o",
```

Используйте код с осторожностью.

Разбор основных шагов

1. **Эмбеддинги (`embeddings.create`)**: Мы не сравниваем слова напрямую. API превращает текст в список из ~1536 чисел, которые отражают его смысл.
2. **Векторный поиск (`faiss`)**: Вместо поиска по ключевым словам, мы ищем в базе вектор, который геометрически ближе всего к вектору вопроса.
3. **Контекстный промпт**: Мы «подсовываем» модели найденный кусок текста, фактически расширяя её кратковременную память этим документом. Хабр +5

Хотите, чтобы я объяснил, как масштабировать это для тысяч документов или как подключить бесплатные локальные модели вместо OpenAI?

да

Для работы с тысячами документов и перехода на локальные модели архитектуру нужно усложнить, заменив простые списки на полноценные базы данных и локальные серверы.

1. Как масштабировать RAG (тысячи и миллионы документов)

Когда данных становится много, поиск «в памяти» (как в примере с FAISS выше) начинает тормозить или потреблять слишком много ОЗУ.

- **Используйте векторные БД (Vector Databases)**: Вместо локальных индексов используйте специализированные решения, поддерживающие шардирование и постоянное хранение.
  - **Qdrant / Milvus / Weaviate**: Open-source лидеры для больших нагрузок.
  - **pgvector**: Расширение для PostgreSQL, если вы хотите хранить векторы рядом с обычными данными.
- **Гибридный поиск (Hybrid Search)**: На больших объемах семантический поиск может ошибаться. Комбинируйте его с классическим полнотекстовым поиском (BM25/Keyword search), чтобы находить точные совпадения имен или артикулов.
- **Metadata Filtering**: Добавляйте метаданные к чанкам (например, `department: "finance"` или `year: 2024`). Это позволяет БД сначала отфильтровать лишнее, а потом искать только среди нужных документов, что в разы ускоряет поиск.
- **Переранжирование (Reranking)**: Поиск в БД выдает топ-10

Режим ИИ Все Видео Покупки Картинки Ещё

Шаг A: Запуск сервера (Ollama или vLLM)

Самый простой способ — **Ollama**. Она запускает модели (Llama 3, Mistral) и предоставляет API на порту `11434`.

7 сайтов YouTube +1

1. Запустите Ollama:
2. Запустите модель: `ollama run llama3`

Retrieval-Augmented Generation (RAG): глубокий ... - Habr
31 июл. 2025 г. — В любом случае, OpenAI задала тон в индустрии: RAG-бот, отвечающий на вопросы по вашей Embeddings-as-a-...

Шаг B: Изменение кода под локальный API

Вам почти не нужно менять код, достаточно указать другой `base_url`.

Хабр
Создаем простую систему RAG на Python / Хабр
29 дек. 2025 г. — ... import RecursiveCharacterTextSplitter from langchain_core.prompts import ChatPromptTemplate from...

python

```python
from openai import OpenAI

# Локальный сервер Ollama имитирует OpenAI API
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama" # ключ не важен, но поле должно быть заполн
)

# Теперь все вызовы client.chat.completions.create
# будут идти на вашу локальную видеокарту
```

Используйте код с осторожностью.

Резюме: Стек для серьезного RAG

Режим ИИ Все Видео Покупки Картинки Ещё

PDF/Word)

Хотите разобрать, как правильно разбивать (chunking) сложные документы, чтобы не терять смысл текста?

да

- **Проблема:** Предложение может разорваться пополам.

- **Recursive Character Splitting (Рекурсивный метод):** Это «золотой стандарт». Алгоритм сначала пытается делить текст по абзацам (`\n\n`), если чанк все еще большой — по предложениям (`.`), затем по словам. Так сохраняется целостность мысли.
- **Semantic Chunking:** Модель анализирует смысл текста и проводит границу там, где тема резко меняется. Это самый точный, но и самый ресурсозатратный способ.

2. Важность "Overlap" (Перекрытия)

Всегда делайте перекрытие между чанками (обычно 10–20% от размера).

- **Зачем:** Если важная информация (например, «Цена составляет... 500 рублей») окажется на стыке, то без перекрытия первый чанк получит «Цена составляет...», а второй — «500 рублей». С перекрытием оба куска сохранят смысл.

3. Практический пример на Python

Для этого лучше всего подходит библиотека `langchain-text-splitters`.

python

```python
from langchain_text_splitters import RecursiveCharacterTextSplit

text = "Ваш очень длинный текст из PDF или базы данных..."

# Настройка разделителя
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,    # Максимальный размер чанка (в символах
```

Используйте код с осторожностью.

4. Продвинутые приемы

- **Markdown/HTML Splitting:** Если ваш документ размечен, лучше делить его по заголовкам (`#`, `##`, `<h3>`). Так модель будет точно знать, что весь текст внутри одного раздела относится к одной теме.
- **Контекстные заголовки:** К каждому чанку в начало можно программно добавлять название документа или краткое резюме всей статьи. Это помогает модели «не теряться», когда она видит вырванный из контекста абзац.

Что выбрать для вашего случая?

1. **Простые инструкции/статьи:** Подойдет `RecursiveCharacterTextSplitter`.
2. **Техническая документация/Код:** Лучше использовать специализированные сплиттеры (например, `MarkdownHeaderTextSplitter` или `PythonCodeTextSplitter`).
3. **Сложные PDF-таблицы:** Обычный текст их «ломает». Тут нужны инструменты вроде **Unstructured** или **LlamaParse**, которые переводят таблицы в формат Markdown перед нарезкой.

Рассказать подробнее про **обработку таблиц в PDF** (это самая большая боль в RAG) или перейдем к **оценке качества** (как понять, что RAG не врет)?

- **LlamaParse**: Считается одним из лучших для сложных PDF. Он сохраняет иерархию заголовков и преобразует таблицы в структурированные Markdown-таблицы, которые LLM понимает идеально.
- **Unstructured**: Мощная библиотека, которая умеет классифицировать каждый элемент (текст, заголовок, список, таблица) и извлекать их по отдельности.
- **Docling**: Новое решение (от IBM), которое конвертирует документы почти идеально и поддерживает интеграцию с другими OCR. Reddit +3

2. Оценка качества: «Триада RAG»

Чтобы понять, что ваш RAG не «галлюцинирует», используется методология **RAG Triad** (Триада RAG). Она проверяет три связи: TruLens

1. **Context Relevance (Релевантность контекста)**: Насколько найденные в базе куски текста вообще относятся к вопросу? Если поиск выдал мусор, LLM не сможет ответить правильно.
2. **Groundedness / Faithfulness (Верность источникам)**: Содержится ли ответ модели *только* в найденных документах? Это проверка на галлюцинации.
3. **Answer Relevance (Релевантность ответа)**: Действительно ли модель ответила на то, что спросил пользователь?. TruLens +1

3. Инструменты для автоматической оценки

Вместо того чтобы проверять тысячи ответов вручную, используют «LLM-как-судью» (LLM-as-a-judge) через специальные фреймворки:

- **Ragas**: Самая популярная библиотека. Она автоматически генерирует тестовые вопросы по вашим документам и выставляет оценки по

Контекст, Ответ), и он вызывает мощную модель (например, GPT-4), чтобы та проанализировала логическую связь между ними. Reddit +1

Хотите, я покажу, как выглядит **итоговая схема архитектуры** профессионального RAG-сервиса со всеми этими компонентами?

да

Режим ИИ Все Видео Покупки Картинки Ещё

Самые сильные команды относятся к оценке RAG так же, как команды...

Хабр

Оценка с помощью Ragas | Документация Milvus

Ragas — это фреймворк, который помогает оценивать конвейеры RAG. Существуют инструменты и фреймворки, которые помогают создавать т...

Milvus

Почему ваш RAG работает неэффективно — DigitalOcean

Переведено — RAG терпит неудачу в основном из-за низкого качества данных и слабой предварительной обработки документов . Неправильное...

DigitalOcean

Показать все

1. **Источники**: PDF, Notion, SQL, Confluence и т.д.
2. **Parsing (LlamaParse / Docling)**: Превращение в Markdown с сохранением структуры таблиц.
3. **Chunking**: Рекурсивное разбиение с перекрытием (overlap).
4. **Embedding Model (локальная BGE или OpenAI)**: Превращение текста в векторы.
5. **Vector DB (Qdrant / Weaviate)**: Хранение векторов вместе с метаданными (дата, автор, категория).

2. **Retrieval Pipeline (Поиск и Генерация)**:
   1. **Query Rewrite**: Модель перефразирует вопрос пользователя для лучшего поиска (например, исправляет опечатки или добавляет контекст из истории чата).
   2. **Hybrid Search**: Поиск в БД одновременно по вектору (смысл) и по ключевым словам (BM25).
   3. **Reranker (BGE-Reranker / Cohere)**: Берет топ-20 результатов и переупорядочивает их, оставляя 3–5 самых точных.
   4. **LLM (GPT-4o / Llama 3)**: Генерирует ответ на основе отфильтрованного контекста.
   5. **Guardrails**: Фильтр, который проверяет ответ на токсичность или раскрытие секретных данных перед показом пользователю.

Визуализация потока данных

mermaid

```
graph TD
    subgraph "1. Подготовка данных (Offline)"
    A[Документы] --> B[LlamaParse: Markdown + Таблицы]
    B --> C[Chunking: 1000 симв + 200 overlap]
```

```
    J --> User
    end

    subgraph "3. Оценка качества"
    I -.-> K[Ragas / TruLens: Анализ триады RAG]
    end
```

Используйте код с осторожностью.

Главные советы по реализации:

1. **Не экономьте на парсинге**: Если таблица превратится в кашу на этапе парсинга, никакой Reranker её не спасет.
2. **Используйте Reranker**: Это самый дешевый и быстрый способ поднять точность RAG на 20-30%.
3. **Логируйте всё**: Сохраняйте, какие именно куски текста были найдены для каждого вопроса. Это позволит вам вручную проверить «слабые места» базы знаний.

Нужно ли помочь с **настройкой конкретной векторной базы данных** (например, Qdrant) или разберем **написание системного промпта**, который запрещает модели выдумывать факты?